

COMP 3700
Software Modeling & Design

Chapter3: State diagram
Dr. Xuechao Li



AUBURN

UNIVERSITY

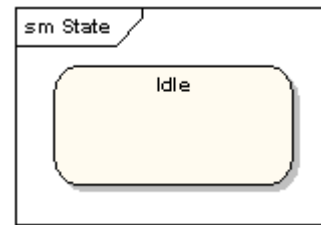
SAMUEL GINN
COLLEGE OF ENGINEERING

UML state diagrams

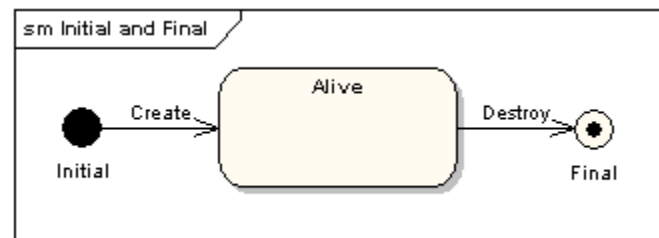
- **state diagram:** Depicts data and behavior of a single object throughout its lifetime.
 - set of states (including an initial start state)
 - transitions between states
 - entire diagram is drawn from that object's perspective

State Machine Diagrams

- **States** - A state is denoted by a round-cornered rectangle with the name of the state written inside it.

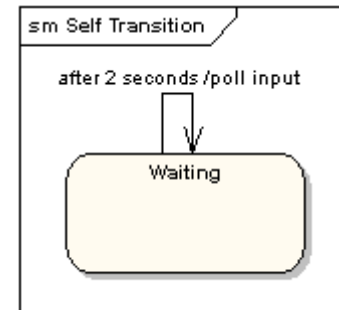
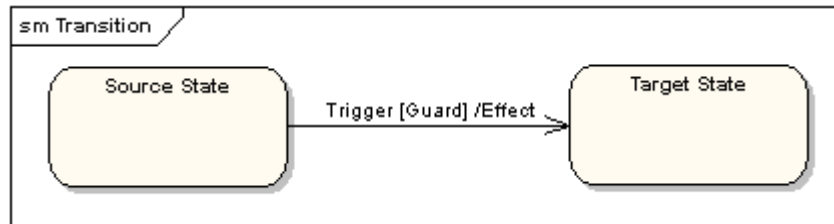


- **Initial and Final States** - The initial state is denoted by a filled black circle and may be labeled with a name. The final state is denoted by a circle with a dot inside and may also be labeled with a name.



State Machine Diagrams

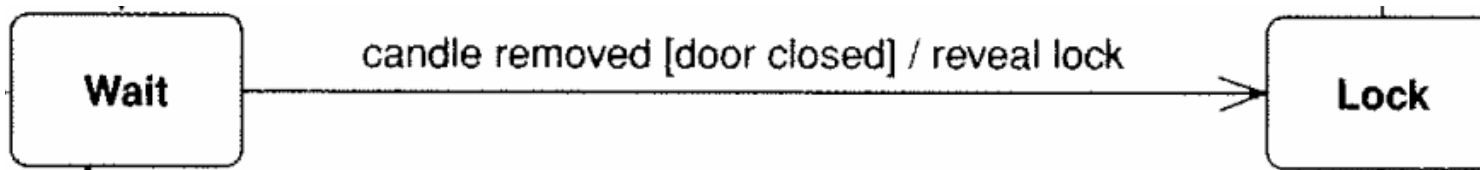
- **Transitions** - Transitions from one state to the next are denoted by lines with arrowheads. A transition may have a trigger, a guard and an effect, as below.
- **Self-Transitions** - A state can have a transition that returns to itself, as in the following diagram. This is most useful when an effect is associated with the transition.



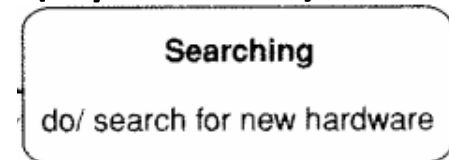
- "**Trigger**" is the cause of the transition, which could be a signal, an event, a change in some condition, or the passage of time. "**Guard**" is a condition which must be true in order for the trigger to cause the transition. "**Effect**" is an action which will be invoked directly on the object that owns the state machine as a result of the transition.

Transitions

- **transition:** movement from one state to another



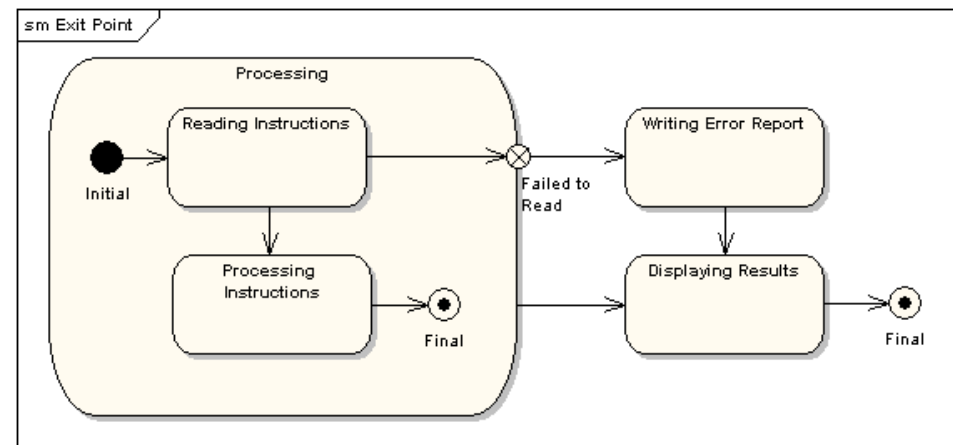
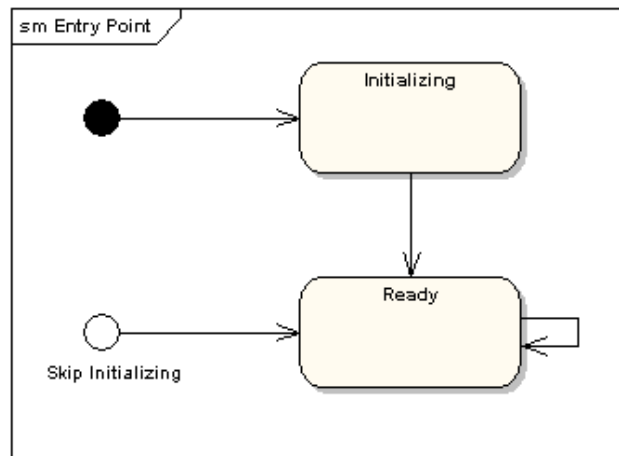
- *signature* [*guard*] / *activity*
 - signature: event that triggers (potential) state change
 - guard: boolean condition that must be true
 - activity: any behavior executed during transition (*optional*)



- transitions must be mutually exclusive (deterministic)
 - must be clear what transition to take for an event

State Machine Diagrams

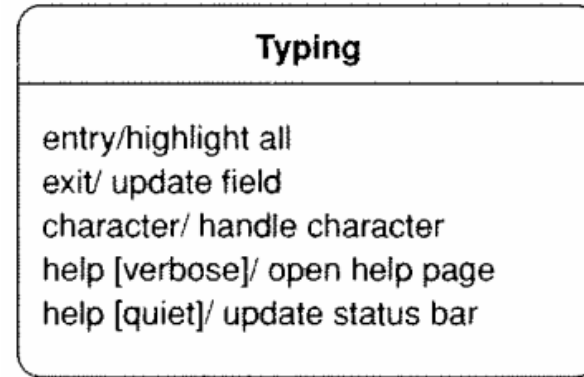
- **Entry Point** - Sometimes you won't want to enter a sub-machine at the normal initial state. For example, in the following sub-machine it would be normal to begin in the "Initializing" state, but if for some reason it wasn't necessary to perform the initialization, it would be possible to begin in the "Ready" state by transitioning to the named entry point.



- **Exit Point** - In a similar manner to entry points, it is possible to have named alternative exit points. The following diagram gives an example where the state executed after the main processing state depends on which route is used to transition out of the state.

Internal activities

- **internal activity:** actions that the central object takes on itself
 - sometimes drawn as self-transitions (events that stay in same state)
- entry/exit activities
 - reasons to start/stop being in that state



State diagram example

